

torch.range#

<https://pytorch.org/docs/stable/generated/torch.range.html#torch.range>

Description:

Returns a 1-D tensor of size $\lfloor \frac{\text{end} - \text{start}}{\text{step}} \rfloor + 1$ with values from start to end with step step. Step is the gap between two values in the tensor. This function is deprecated and will be removed in a future release because its behavior is inconsistent with Python's range builtin. Instead, use torch.arange(), which produces values in [start, end). start (float, optional) – the starting value for the set of points. Default: 0. end (float) – the ending value for the set of points

Function Signature

```
torch.range(start=0, end, step=1, *, out=None, dtype=None, layout=torch.strided, device=None, requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor. dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see torch.set_default_dtype()). If dtype is not given, infer the data type from the other input arguments. If any of start, end, or step are floating-point, the dtype is inferred to be the default dtype, see get_default_dtype(). Otherwise, the dtype is inferred to be torch.int64. layout (torch.layout, optional) – the desired layout of returned Tensor. Default: torch.strided. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensors

Code Examples

```
>>> torch.range(1, 4)
tensor([ 1.,  2.,  3.,  4.])
>>> torch.range(1, 4, 0.5)
tensor([ 1.0000,  1.5000,  2.0000,  2.5000,  3.0000,  3.5000,
         4.0000])
```

Notes & Warnings

WARNING

This function is deprecated and will be removed in a future release because its behavior is inconsistent with Python's `range` builtin. Instead, use `torch.arange()`, which produces values in `[start, end)`.

Detailed Documentation

Documentation

Returns a 1-D tensor of size $\lfloor \frac{\text{end} - \text{start}}{\text{step}} \rfloor + 1$ with values from `start` to `end` with `step` step. Step is the gap between two values in the tensor.

This function is deprecated and will be removed in a future release because its behavior is inconsistent with Python's `range` builtin. Instead, use `torch.arange()`, which produces values in `[start, end)`.

`start` (float, optional) – the starting value for the set of points. Default: 0.

`end` (float) – the ending value for the set of points

`step` (float, optional) – the gap between each pair of adjacent points. Default: 1.

`out` (Tensor, optional) – the output tensor.

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None,

uses a global default (see `torch.set_default_dtype()`). If `dtype` is not given, infer the data type from the other input arguments. If any of `start`, `end`, or `step` are floating-point, the `dtype` is inferred to be the default `dtype`, see `get_default_dtype()`. Otherwise, the `dtype` is inferred to be `torch.int64`.

`layout` (`torch.layout`, optional) – the desired layout of returned Tensor. Default: `torch.strided`.

`device` (`torch.device`, optional) – the desired device of returned tensor. Default: if `None`, uses the current device for the default tensor type (see `torch.set_default_device()`). `device` will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`requires_grad` (`bool`, optional) – If autograd should record operations on the returned tensor. Default: `False`.